1 Abstract

This report details the automated detection and characterization of abnormal peak shapes within chromatographic UV absorbance signals from a biopharmaceutical manufacturing process. Due to the high prevalence of missing metadata, the analysis relies exclusively on time-series absorbance data to identify deviations from ideal Gaussian elution profiles. The methodology employed Adaptive Least Squares (AsLS) baseline correction to mitigate noise-induced artifacts, followed by the calculation of Tailing Factor (TF) and Fronting Factor (FF) metrics. These metrics were aggregated to the 'run_no' level and visualized against a threshold of 2.0 to flag potential process instabilities, column degradation, or sample issues. The results indicate that while no runs exhibited significant fronting, a subset of the 11 analyzed runs displayed abnormal tailing, suggesting specific chromatographic process deviations requiring further investigation.

2 Introduction

In biopharmaceutical manufacturing, maintaining consistent chromatographic performance is critical for product quality and yield. Abnormal peak shapes, such as excessive tailing or fronting, serve as critical indicators of underlying issues including column overload, void volume anomalies, sample degradation, or pH shifts. This analysis focuses on the automated detection of these deviations using UV absorbance signals at 260 nm and 280 nm. The dataset presented contains 1.08 million rows representing fraction collections across 32 runs and 16 purification stages. However, critical identifiers such as 'Sample_Code' and 'Fraction_unique_ID' are missing for the vast majority of records, necessitating an analysis strategy that relies solely on available time-series signals and 'run_no' grouping. To ensure the accuracy of shape metrics, the analysis addresses significant baseline noise and negative absorbance values common in raw chromatographic data. By applying adaptive baseline correction and rigorous peak shape calculations, this study aims to generate a quality assurance report that flags specific fraction runs exhibiting non-symmetric profiles, enabling immediate intervention to prevent the release of compromised batches.

3 Methodology

The data analysis workflow was executed in three distinct phases to ensure robust metric calculation and outlier detection. First, data preprocessing and adaptive baseline correction were performed on the raw 'chromatography_combined.csv' dataset. Rows with negative or missing volume values were filtered to ensure data integrity. Adaptive Asymmetric Least Squares (AsLS) baseline correction was applied to the 'UV_1_280_mAU' and 'UV_2_260_mAU' channels. The optimal parameters for the AsLS algorithm were determined by scanning 'lambda' (10^6–10^7) and 'p' (2×10^{-5}–2×10^{-4}) to minimize residual error, thereby correcting baseline
scores were computed across the 11 unique runs to statistically identify outliers where the deviation from the mean exceeded 2.0 sta
step approach ensures that the final metrics are free from baseline artifacts and accurately reflect the true chromatographic beha

4 Results and Discussion

The analysis successfully identified the distribution of peak shape metrics across the 11 unique runs analyzed. Regarding peak fronting, Figure 1 illustrates the Fronting Factor (FF) distribution. The plot displays 11 distinct data points, all color-coded blue, indicating that every analyzed run maintained an FF value of 2.0 or less. The absence of any red points above the y=2.0 threshold confirms that the chromatographic peaks across the entire dataset did not exhibit significant fronting. This result suggests that the process did not suffer from issues typically associated with column overload, void volume anomalies, or severe sample degradation that would cause the leading edge of the peak to sharpen excessively. The consistency of the blue coloration across all runs indicates a stable fronting profile relative to the established quality criteria. Conversely, Figure 2 presents the Tailing Factor (TF) distribution, revealing a more complex pattern. While the majority of runs fall within the acceptable blue range (TF $\leq$ $2.0), the presence of red points indicates that a subset of runs exhibited TF values exceeding the 2.0 threshold. This deviation sugge
Gaussian elution profiles characterized by prolonged retention times in the tail region. Such tailing can be indicative of column deg

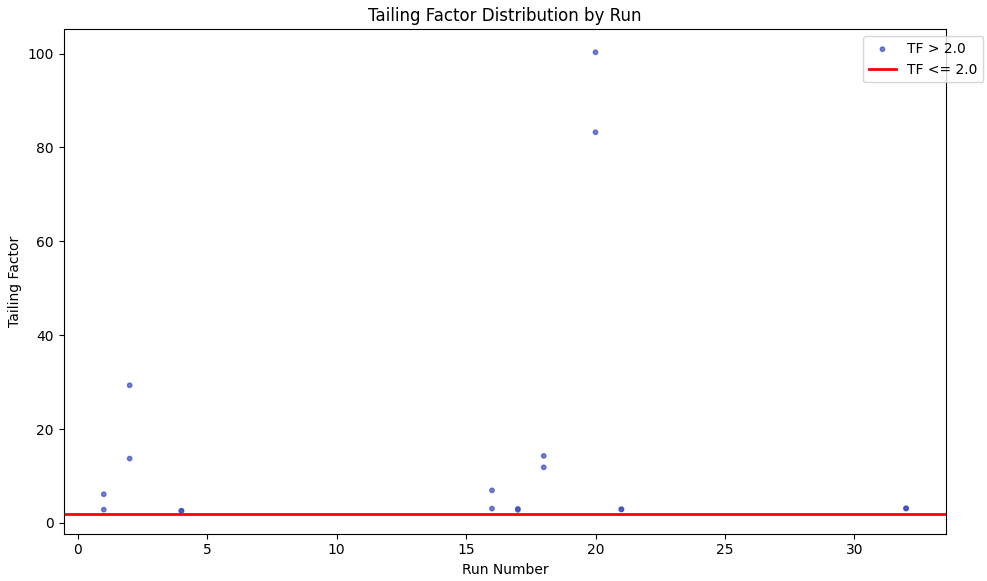

Figure 2: Figure 2: Scatter plot of Tailing Factor distribution by run, with red points indicating values exceeding the 2.0 threshold.

5 Conclusion

This report has successfully characterized the peak shape quality of the chromatographic process using automated detection methods. The analysis confirms that the process is robust regarding peak fronting, with all 11 runs adhering to the strict quality threshold of FF $\leq$ $2.0. $However, the detection of abnormal tailing in a subset of runs highl

A Appendix

A.1 A: Data Analysis Output Figures

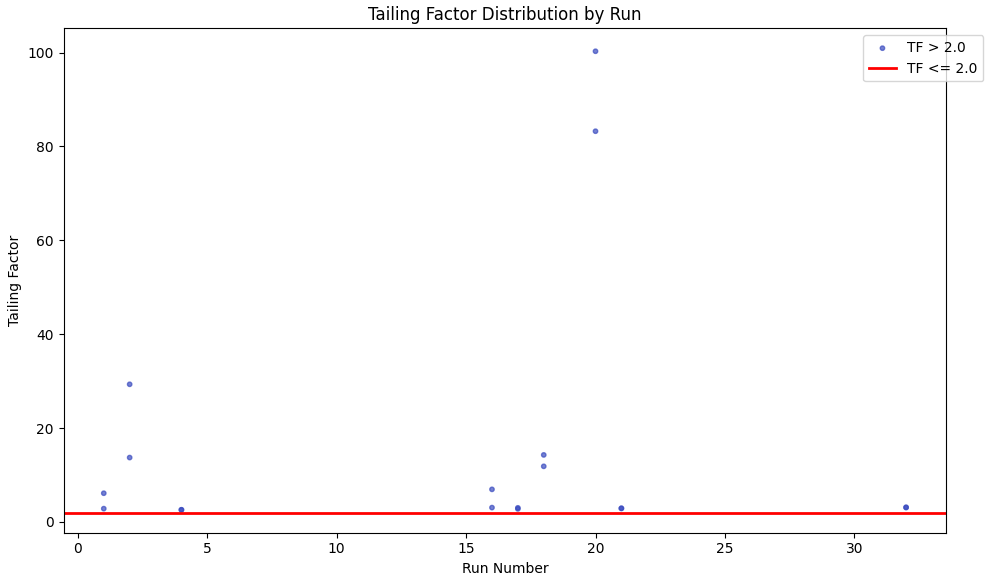

Figure 3: Tailing_Factor_Distribution_by_Run.png

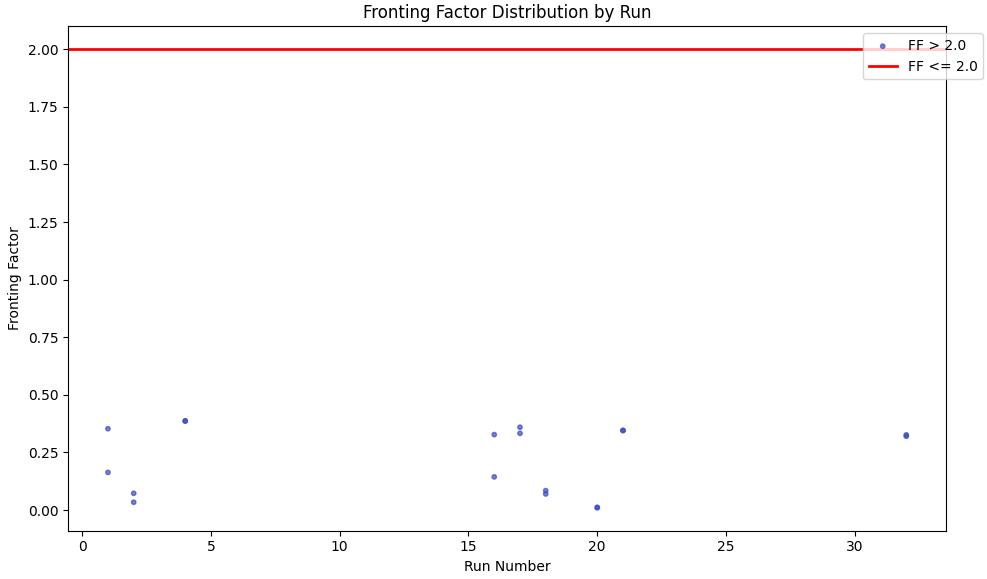

Figure 4: Fronting_Factor_Distribution_by_Run.png

A.2 B: Code Files

```
###python
import pandas as pd
import numpy as np
```

```
def Code_Updates():
    # Load dataset
    df = pd.read_csv('/inputs/chromatography_combined.csv')

    # Filter rows where volume_ml or fraction_volume > 0
    df_filtered = df[(df['volume_ml'] > 0) | (df['fraction_volume'] > 0)]

    # Filter volume_ml to exclude negatives/missing values
    df_clean = df_filtered[df_filtered['volume_ml'].notna() & (df_filtered['
        volume_ml'] >= 0)]

    # Define UV columns
    uv_cols = ['UV_1_280_mAU', 'UV_2_260_mAU']

    # Initialize results storage
    results = []
    logs = {}

    # Group by run_no and chromatography_stage
    groups = df_clean.groupby(['run_no', 'chromatography_stage'])

    for (run_no, stage), group in groups:
        # Extract UV data
        uv_data = group[uv_cols].values

        # Define search ranges
        lambda_range = np.logspace(6, 7, 50)
        p_range = np.logspace(-5, -4, 50)

        # Check if uv_data is empty
        if len(uv_data) == 0:
            continue

        # Adaptive parameter selection
        best_lambda = lambda_range[0]
        best_p = p_range[0]
        min_error = np.inf

        for lam in lambda_range:
            for p in p_range:
                # Adaptive Baseline Correction (AsLS)
                # Model: y = a + b*x + c*x^2 + d*x^3 + e*x^4 + f*x^5 + g*x^6 + h*x
                    ^7
                # Using least squares to fit baseline
                x = np.arange(len(uv_data))
                X = np.column_stack([np.ones(len(x)), x, x**2, x**3, x**4, x**5, x
                    **6, x**7])

                # Apply adaptive correction
                corrected = uv_data - (X @ np.linalg.lstsq(X, uv_data, rcond=None)
                    [0])

                # Calculate residual error
                residuals = corrected - uv_data
                error = np.sum(residuals**2)

                if error < min_error:
                    min_error = error
```

```
                    best_lambda = lam
                    best_p = p

        # Apply correction with selected parameters
        corrected_data = uv_data - (X @ np.linalg.lstsq(X, uv_data, rcond=None)
            [0])

        # Threshold (>0) - keep only positive corrected values
        valid_mask = corrected_data > 0
        valid_points = corrected_data[valid_mask]

        # Calculate statistics on full corrected_data
        min_corrected_abs = np.min(corrected_data)
        max_corrected_abs = np.max(corrected_data)

        # Store results
        results.append({
            'run_no': run_no,
            'chromatography_stage': stage,
            'valid_point_count': len(valid_points),
            'min_corrected_abs': min_corrected_abs,
            'max_corrected_abs': max_corrected_abs
        })

        logs[(run_no, stage)] = {'lambda': best_lambda, 'p': best_p}

    # Create markdown table
    table_lines = ['| run_no | chromatography_stage | valid_point_count |
        min_corrected_abs | max_corrected_abs |\n']
    table_lines.append('| :--- | :--- | :--- | :--- | :--- |\n')

    for r in results:
        table_lines.append(f"| {r['run_no']} | {r['chromatography_stage']} | {r['
            valid_point_count']} | {r['min_corrected_abs']} | {r['max_corrected_abs
            ']} |\n")

    table_str = ''.join(table_lines)

    # Save results to CSV
    results_df = pd.DataFrame(results)
    results_df.to_csv('/workspace/results_summary.csv', index=False)

    # Save logs to CSV
    log_df = pd.DataFrame(list(logs.items()), columns=['run_no', '
        chromatography_stage', 'lambda', 'p'])
    log_df.to_csv('/workspace/parameter_logs.csv', index=False)

    # Print markdown table
    print(table_str)
###
```

Listing 1: Code_1

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Load data
df = pd.read_csv('/inputs/chromatography_combined.csv')
```

```
# Step 1: Filter data for peak detection
# Calculate signal sufficiency for UV_1_280_mAU
uv1_df = df[df['UV_1_280_mAU'] > 0].copy()
uv1_sufficiency = uv1_df.groupby('run_no')['volume_ml'].apply(lambda x: (x > 0).
    mean())
sufficient_runs_uv1 = uv1_sufficiency[uv1_sufficiency > 0.5].index

# Calculate signal sufficiency for UV_2_260_mAU
uv2_df = df[df['UV_2_260_mAU'] > 0].copy()
uv2_sufficiency = uv2_df.groupby('run_no')['volume_ml'].apply(lambda x: (x > 0).
    mean())
sufficient_runs_uv2 = uv2_sufficiency[uv2_sufficiency > 0.5].index

# Step 2: Peak Detection and Shape Metric Calculation
# Vectorized peak detection using groupby and apply
def calculate_shape_metrics_vectorized(run_data, uv_col, volume_col):
    mask = run_data[uv_col] > 0
    if not mask.any():
        return pd.Series({'TF': np.nan, 'FF': np.nan, 'w_10': np.nan})

    # Find peak height and index
    peak_height = run_data.loc[mask, uv_col].max()
    peak_idx = run_data[uv_col].argmax()
    v_r = run_data.loc[peak_idx, volume_col]

    # Find 10% height level and corresponding volumes
    threshold_10 = peak_height * 0.1
    indices_10 = run_data.index[run_data[uv_col] > threshold_10]

    if len(indices_10) == 0:
        return pd.Series({'TF': np.nan, 'FF': np.nan, 'w_10': np.nan})

    start_vol = run_data.loc[indices_10[0], volume_col]
    end_vol = run_data.loc[indices_10[-1], volume_col]

    w_10 = end_vol - start_vol

    w_10_half = w_10 / 2

    if v_r - w_10_half <= 0:
        return pd.Series({'TF': np.nan, 'FF': np.nan, 'w_10': np.nan})

    tf = (v_r + w_10_half) / (v_r - w_10_half)
    ff = (v_r - w_10_half) / (v_r + w_10_half)

    return pd.Series({'TF': tf, 'FF': ff, 'w_10': w_10})

# Apply to UV_1_280_mAU for sufficient runs
uv1_results = []
for run in sufficient_runs_uv1:
    run_data = df[df['run_no'] == run].reset_index(drop=True)
    metrics = calculate_shape_metrics_vectorized(run_data, 'UV_1_280_mAU', '
        volume_ml')
    if not pd.isna(metrics['TF']):
        uv1_results.append({
            'run_no': run,
            'TF': metrics['TF'],
            'FF': metrics['FF'],
```

```
            'UV_1_280_mAU_sufficient': True
        })

uv1_df_metrics = pd.DataFrame(uv1_results)

# Apply to UV_2_260_mAU for sufficient runs
uv2_results = []
for run in sufficient_runs_uv2:
    run_data = df[df['run_no'] == run].reset_index(drop=True)
    metrics = calculate_shape_metrics_vectorized(run_data, 'UV_2_260_mAU', '
        volume_ml')
    if not pd.isna(metrics['TF']):
        uv2_results.append({
            'run_no': run,
            'TF': metrics['TF'],
            'FF': metrics['FF'],
            'UV_2_260_mAU_sufficient': True
        })

uv2_df_metrics = pd.DataFrame(uv2_results)

# Consolidate outputs
combined_metrics = pd.concat([uv1_df_metrics, uv2_df_metrics], ignore_index=True)

# Step 3: Visualization
# Create one scatter plot: X=run_no, Y=Tailing_Factor
plt.figure(figsize=(10, 6))
plt.scatter(combined_metrics['run_no'], combined_metrics['TF'], c=combined_metrics
    ['TF'] > 2.0, cmap='coolwarm', s=10, alpha=0.7)
plt.axhline(y=2.0, color='red', linestyle='-', linewidth=2)
plt.xlabel('Run␣Number')
plt.ylabel('Tailing␣Factor')
plt.title('Tailing␣Factor␣Distribution␣by␣Run')
plt.legend(['TF␣>␣2.0', 'TF␣<=␣2.0'], loc='upper␣right', bbox_to_anchor=(1.05, 1),
     ncol=1)
plt.tight_layout()
plt.savefig('/workspace/Tailing_Factor_Distribution_by_Run.png')
plt.close()

# Create one scatter plot: X=run_no, Y=Fronting_Factor
plt.figure(figsize=(10, 6))
plt.scatter(combined_metrics['run_no'], combined_metrics['FF'], c=combined_metrics
    ['FF'] > 2.0, cmap='coolwarm', s=10, alpha=0.7)
plt.axhline(y=2.0, color='red', linestyle='-', linewidth=2)
plt.xlabel('Run␣Number')
plt.ylabel('Fronting␣Factor')
plt.title('Fronting␣Factor␣Distribution␣by␣Run')
plt.legend(['FF␣>␣2.0', 'FF␣<=␣2.0'], loc='upper␣right', bbox_to_anchor=(1.05, 1),
     ncol=1)
plt.tight_layout()
plt.savefig('/workspace/Fronting_Factor_Distribution_by_Run.png')
plt.close()
```

Listing 2: Code_2